

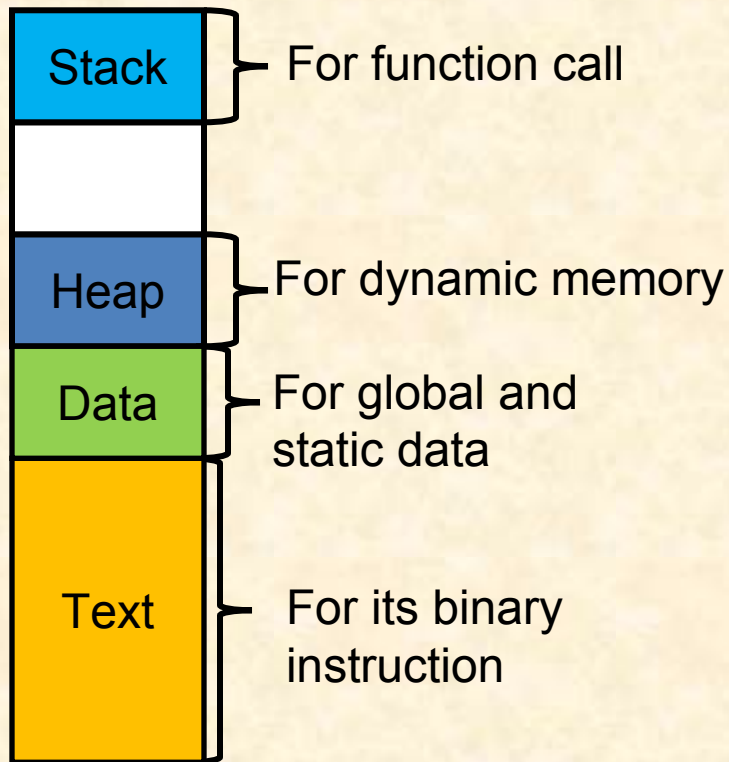
Virtual Memory

Dr. Sk Subidh Ali

IIT Bhilai

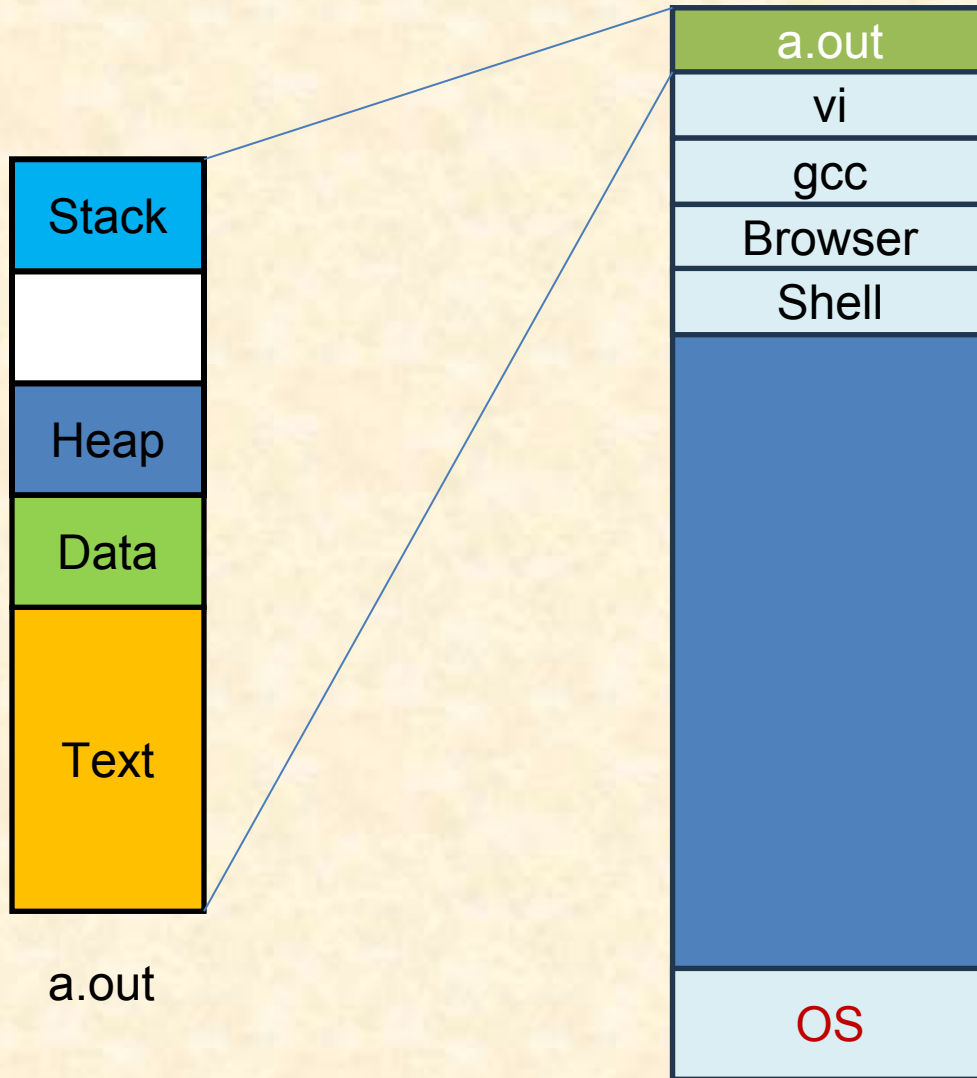
Process memory

Process needs four chunk of memory



a.out

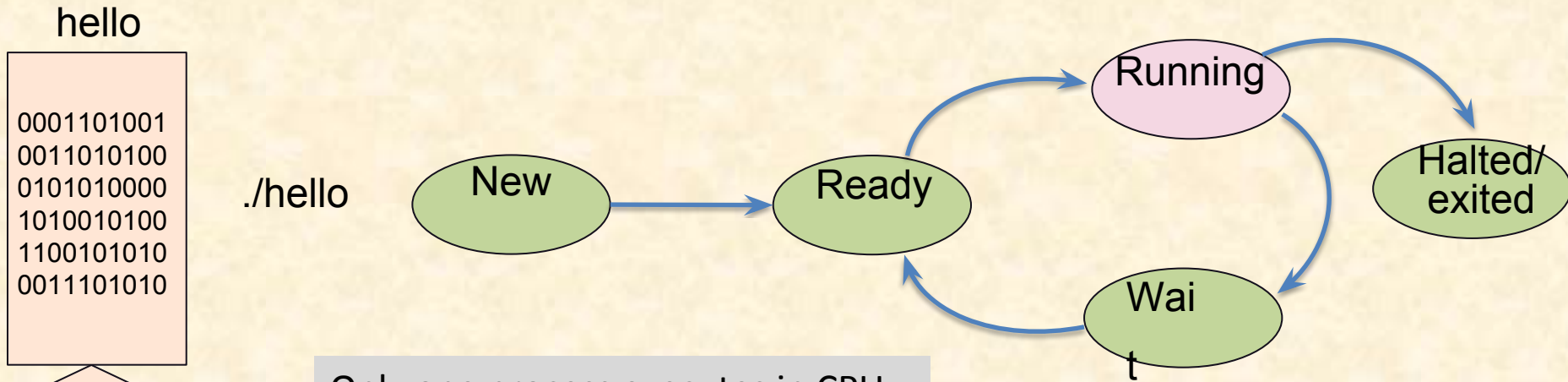
Sharing RAM in multiprogramming



- Multiprogramming

Multiprogramming Environment

Executes multiple programs together



Only one process executes in CPU

`$gcc hello.c -o hello`

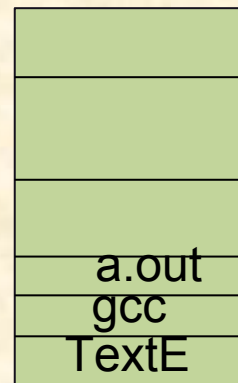
Process must be in RAM for execution

a.c

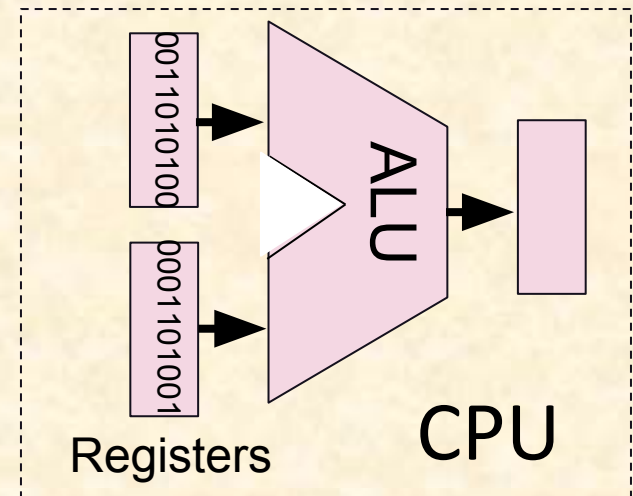
```
Void main()
{
  int a ,b=9,c=8;
  a=b+c
  print(a)
}
```

Need:

- Ready queue
- Waiting queue



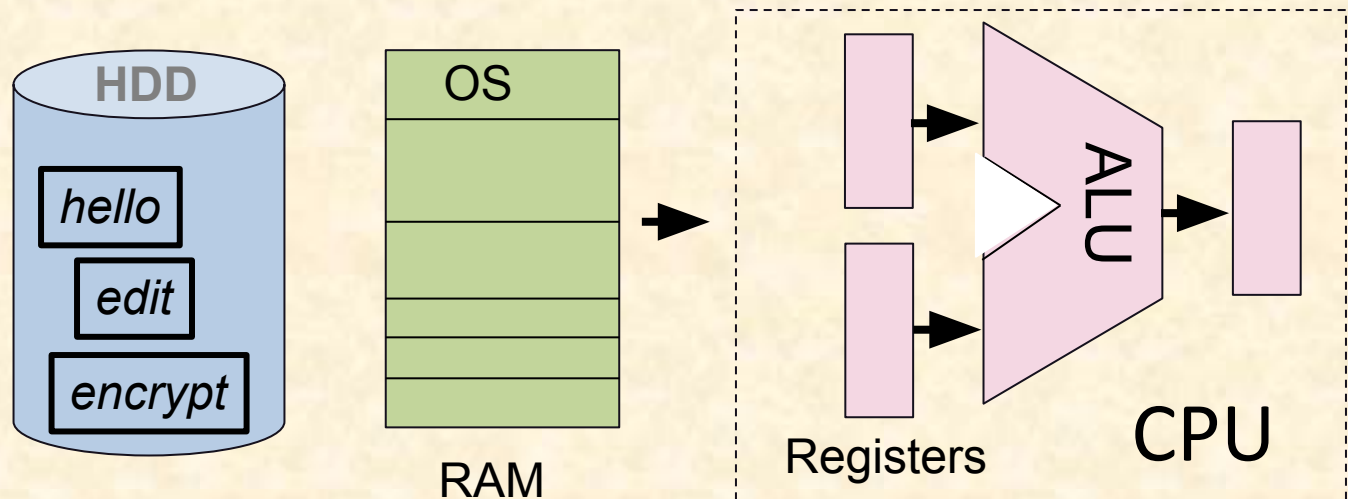
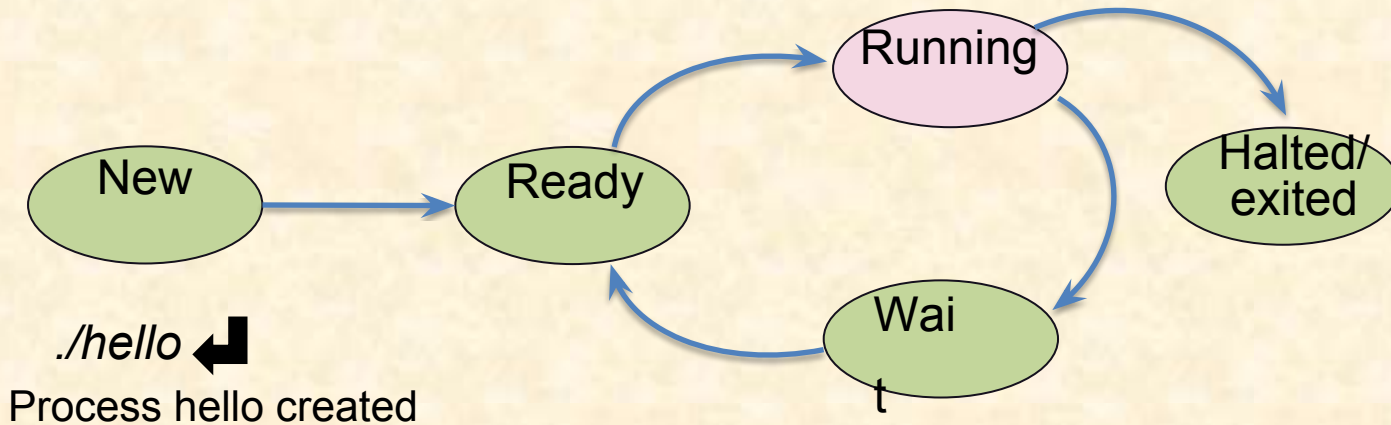
RAM



A running program is called a

Multiprogramming Environment

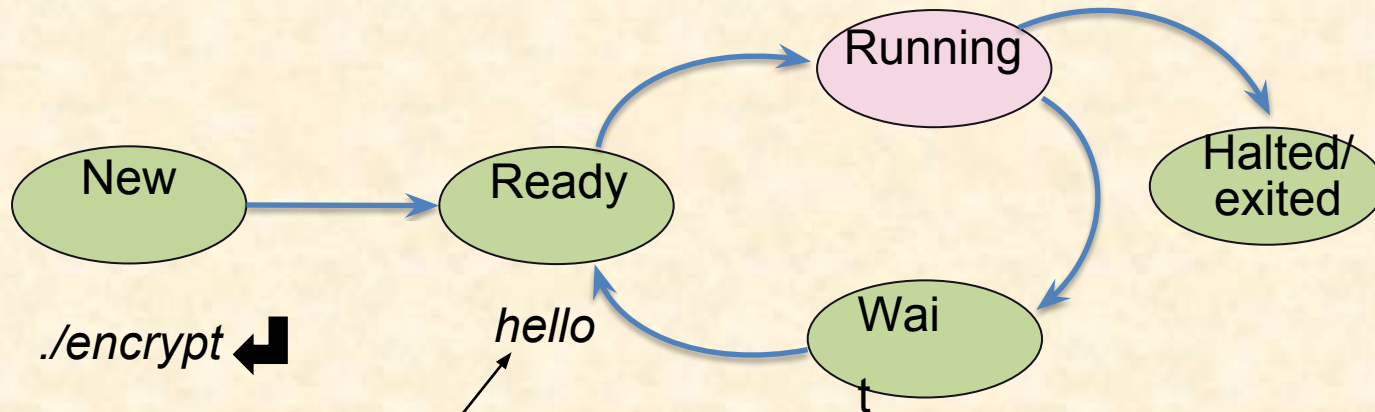
Executes multiple programs together



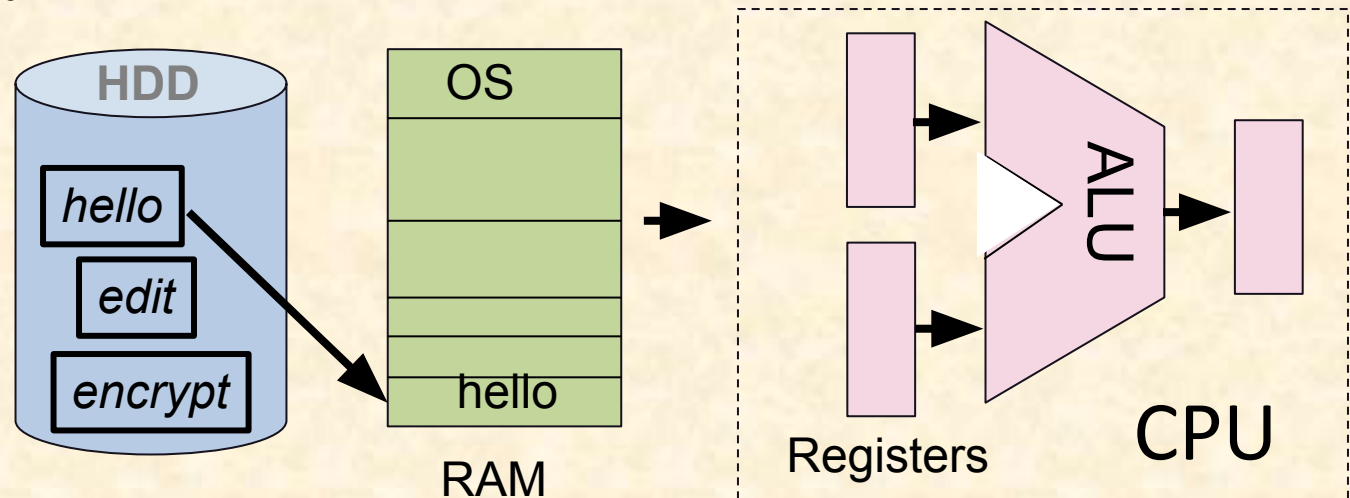
HDD : hard disc

Multiprogramming Environment

Executes multiple programs together

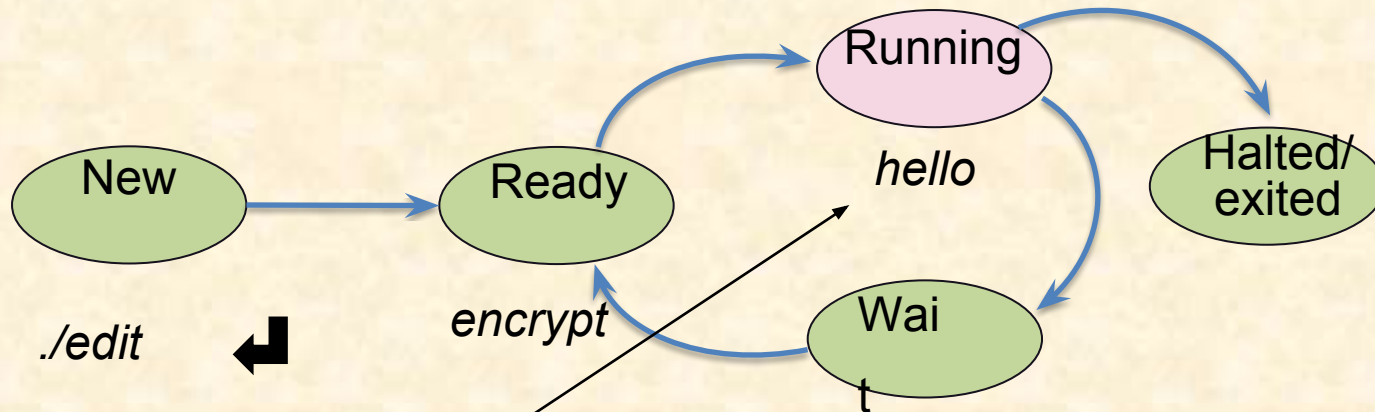


Entire code of hello
is loaded in RAM

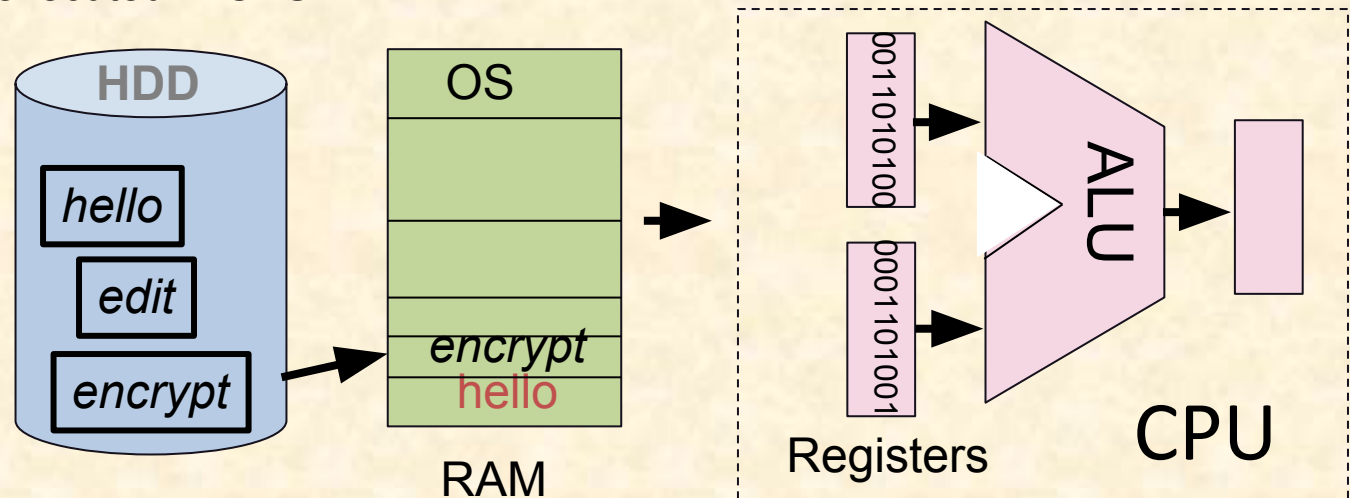


Multiprogramming Environment

Executes multiple programs together

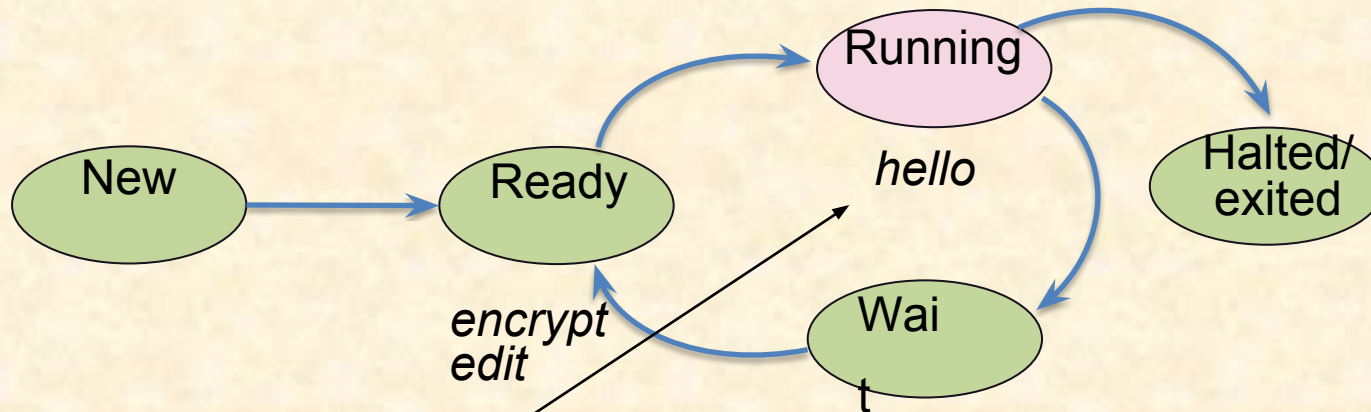


Instructions of hello are getting executed in CPU



Multiprogramming Environment

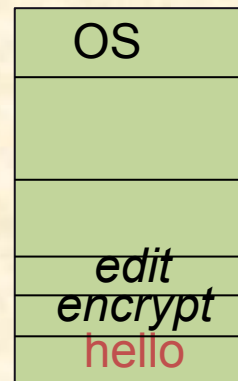
Executes multiple programs together



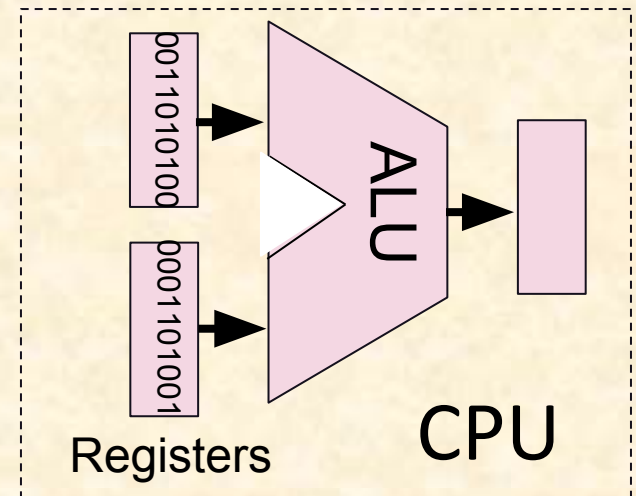
Instructions of hello is getting executed in CPU

```
Void main()
{
  int a= ,b=6,
    c=8;
  scan(a)
  c=a+b+c
  print(a)
  print(c)
}
```

- I/O operations executes in I/O devices (slow)
- take more time



RAM

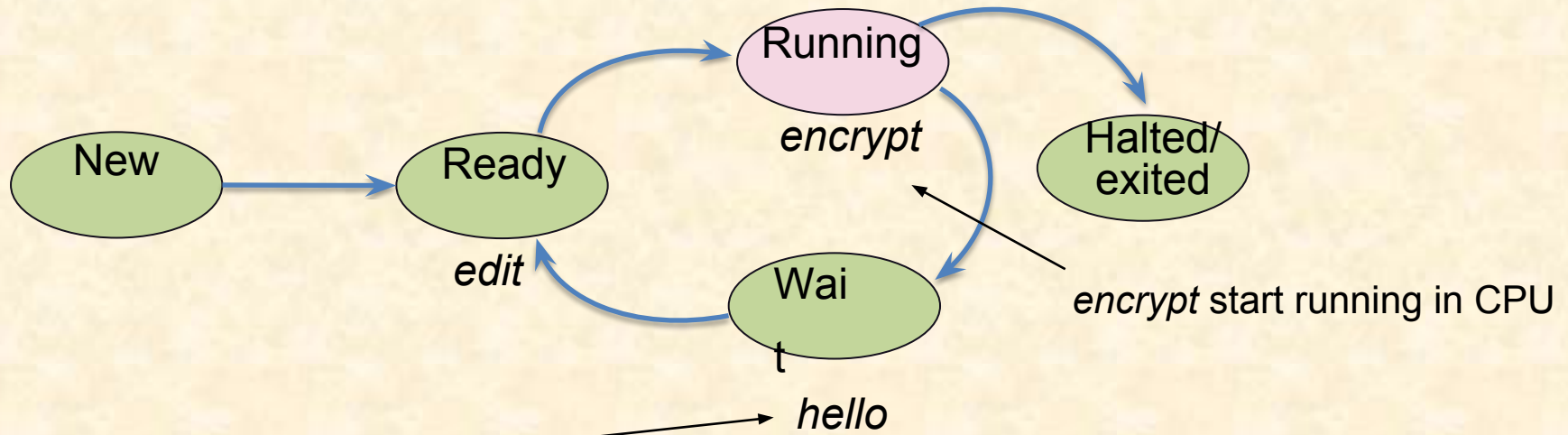


Registers

CPU

Multiprogramming Environment

Executes multiple programs together



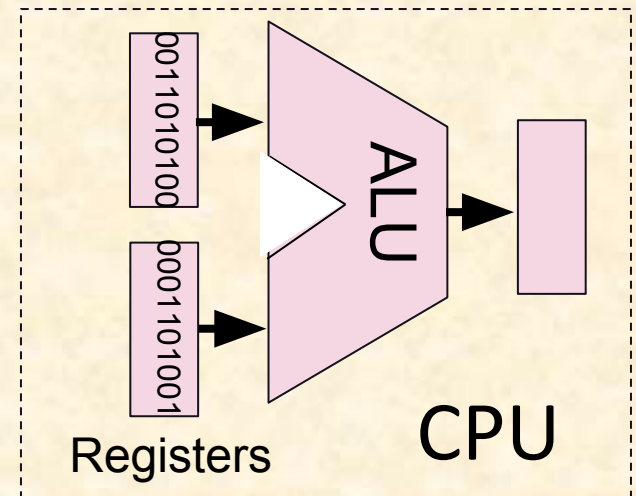
hello is waiting for I/O
(scan(a) to complete)

```
Void main()
{
  int a= ,b=6,
    c=8;
  scan(a)
  c=a+b+c
  print(a)
  print(c)
}
```

- I/O operations executes in I/O devices (slow)
- take more time



RAM

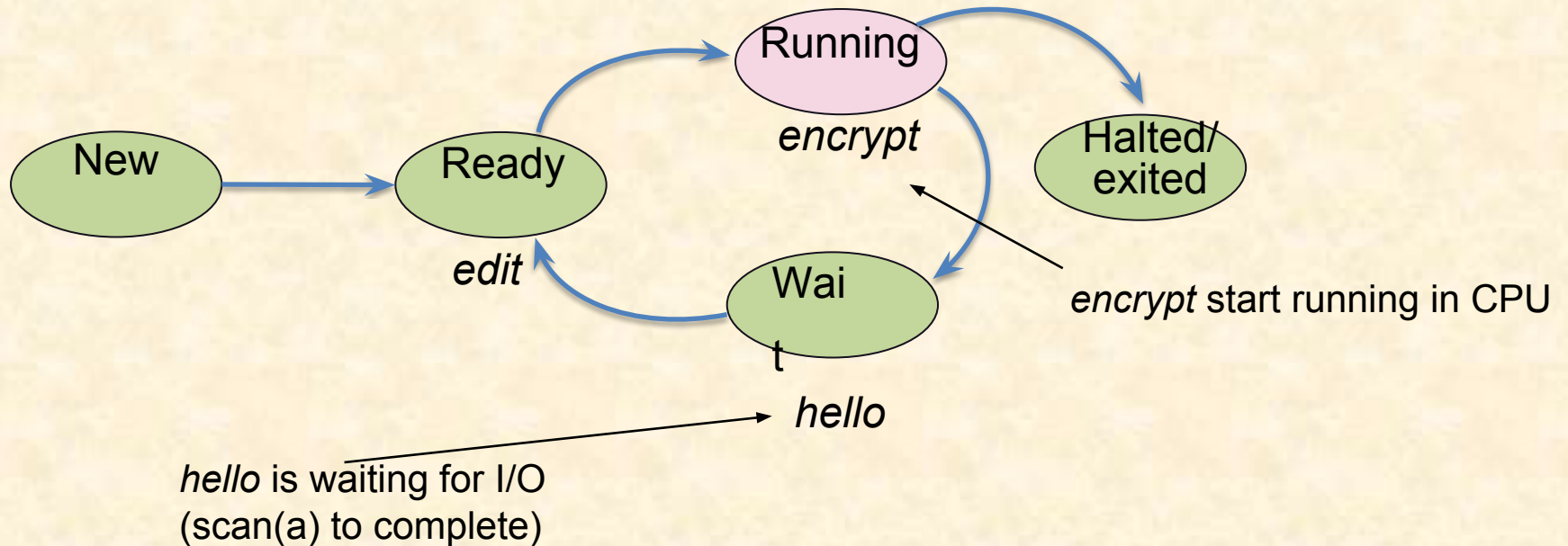


Registers

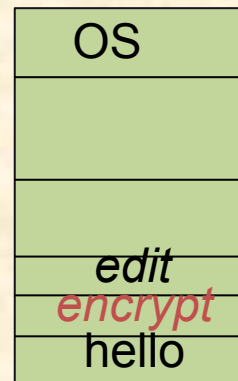
CPU

Multiprogramming Environment

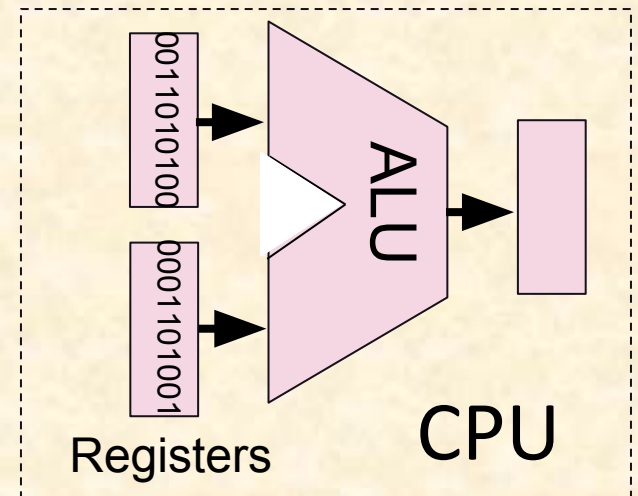
Executes multiple programs together



Memory should be partitioned for different processes



RAM



Issues in sharing physical memory

- Protection

- Must protect process A from corrupting process B
- Maintain privacy

- Transparency

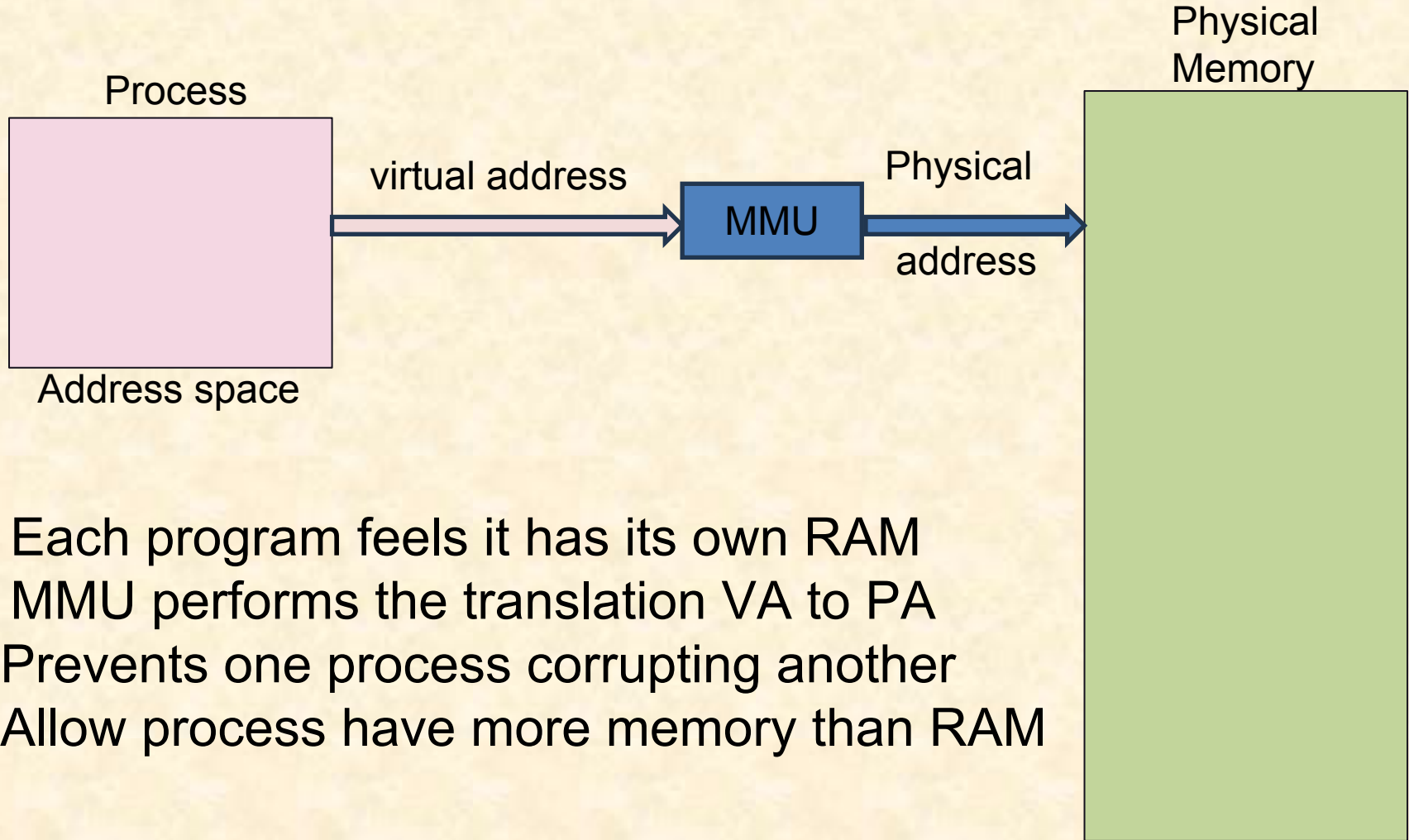
- A process shouldn't require particular memory locations
- Processes often require large amounts of contiguous memory (for stack, large data structures, etc.)

- Resource exhaustion

- Programmers typically assume machine has “enough” memory
- Sum of sizes of all processes often greater than physical memory

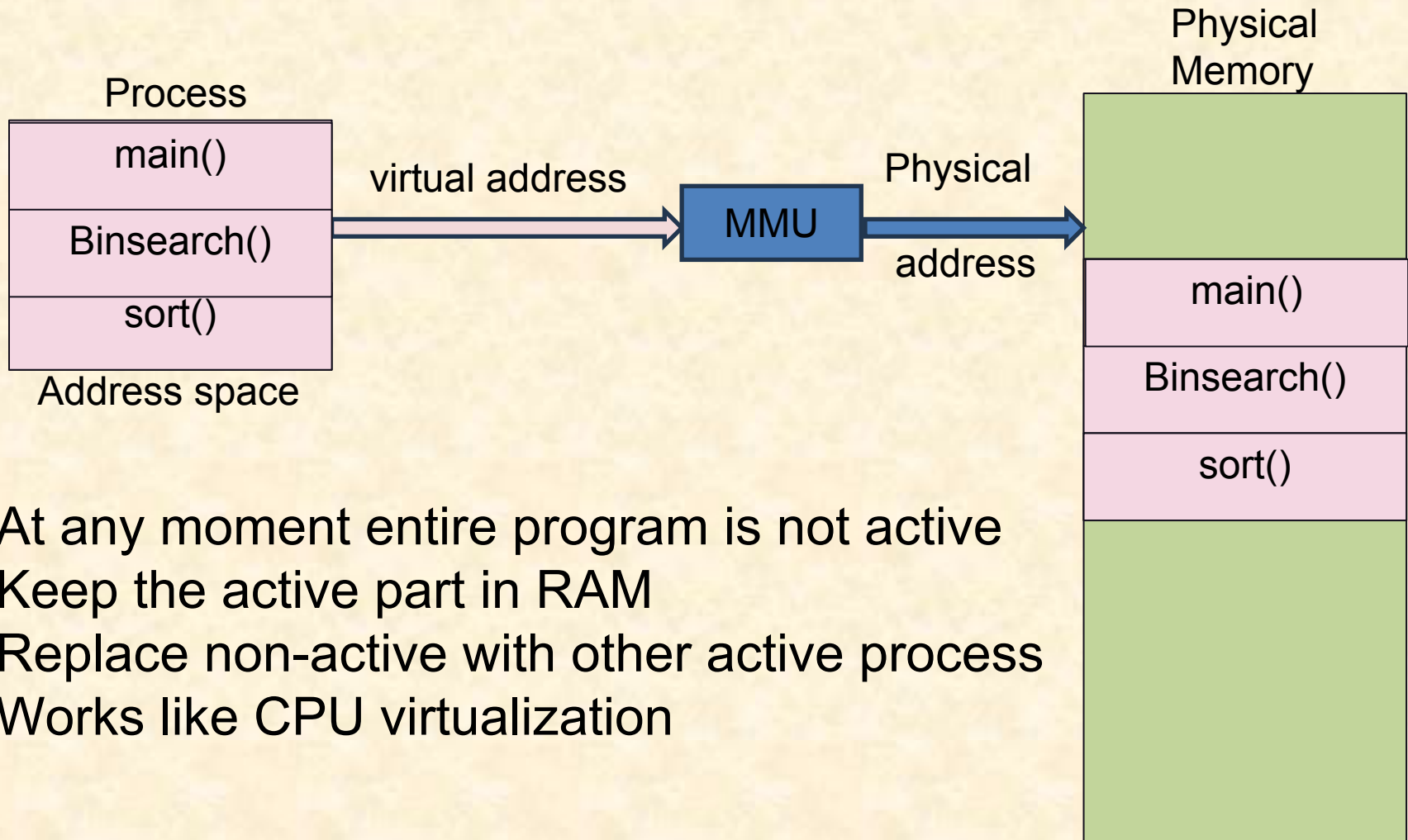
Virtual memory

Virtual memory goal



- Each program feels it has its own RAM
- MMU performs the translation VA to PA
- Prevents one process corrupting another
- Allow process have more memory than RAM

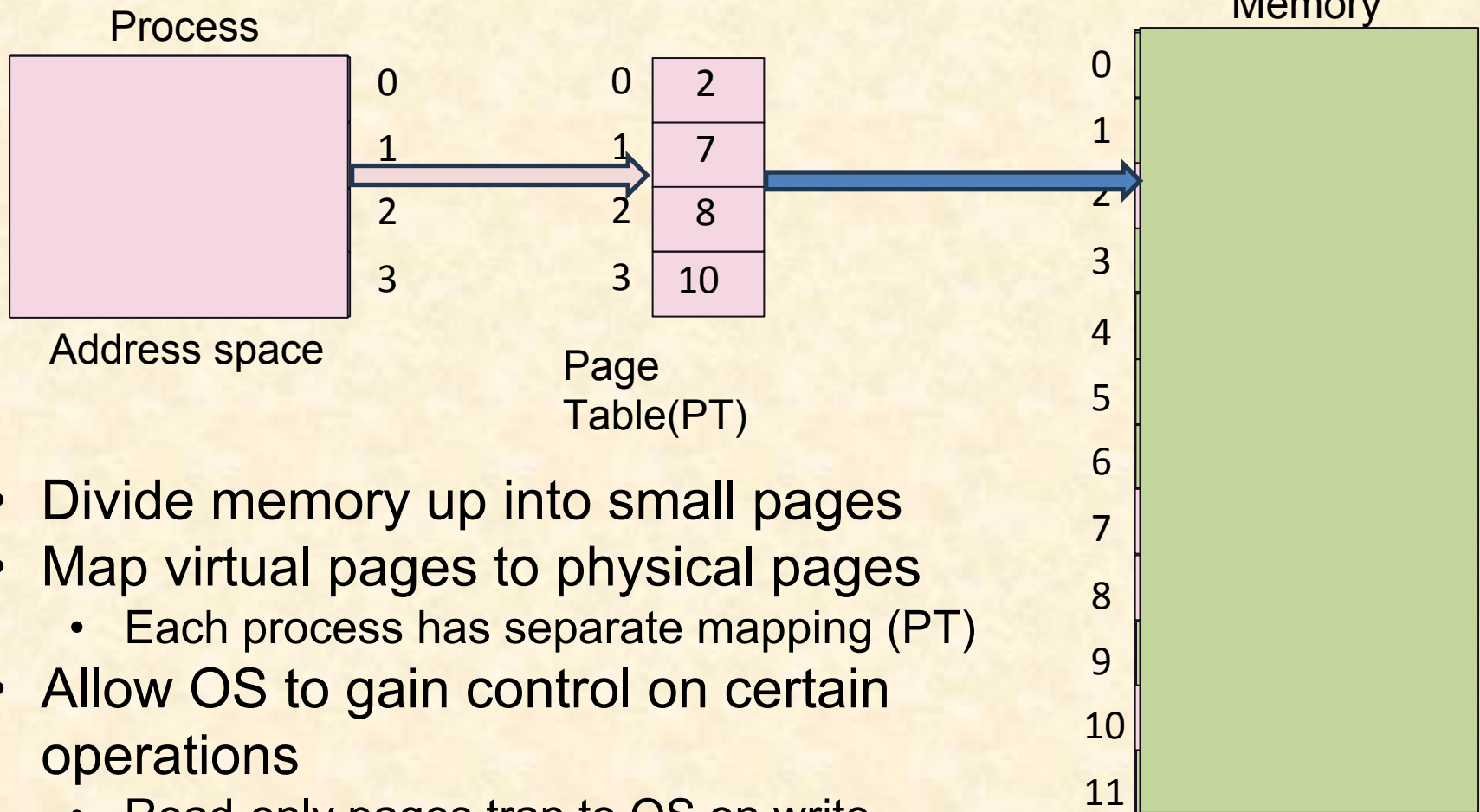
Active vs Passive part of a process



- At any moment entire program is not active
- Keep the active part in RAM
- Replace non-active with other active process
- Works like CPU virtualization

Paging

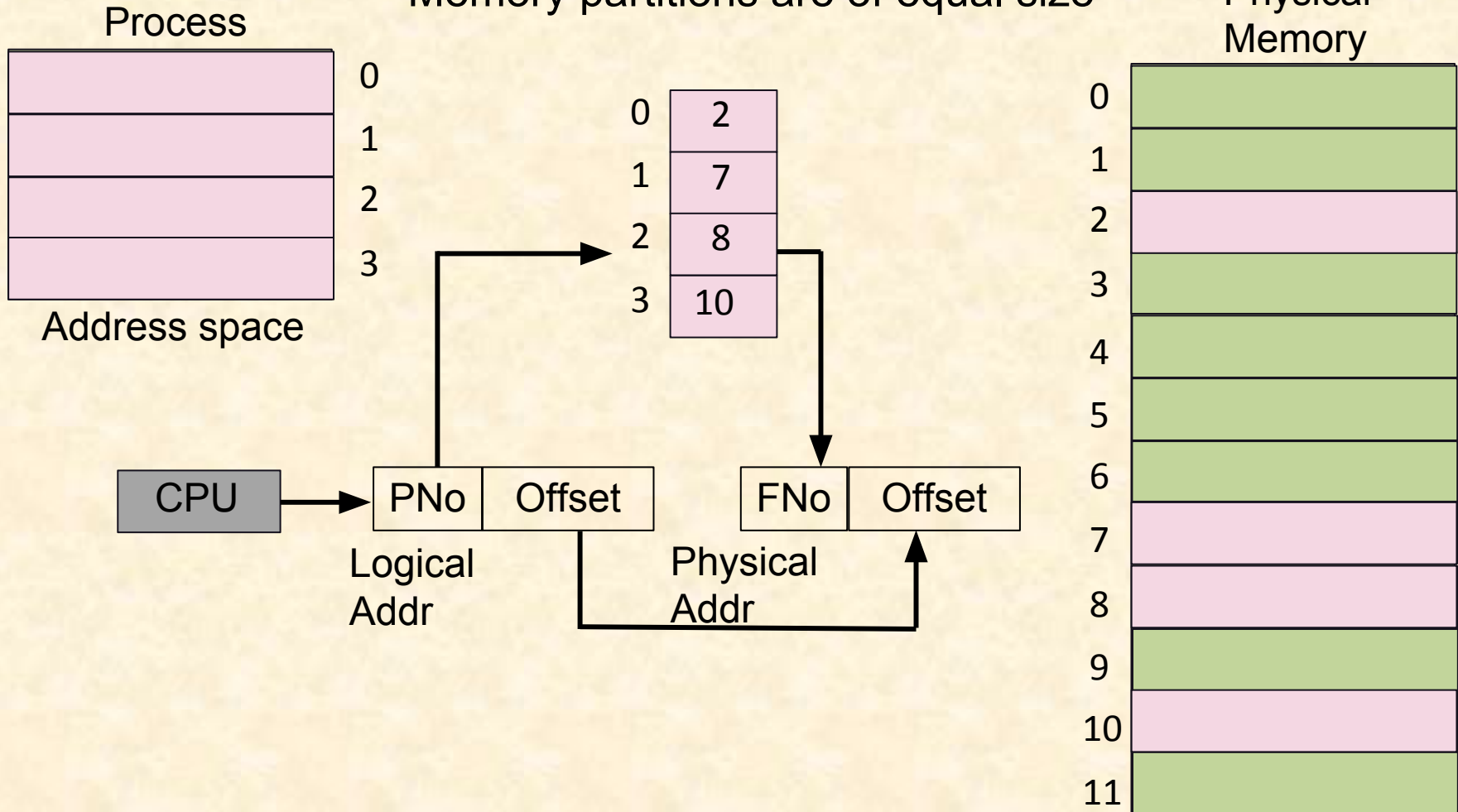
Typical page size is 4KB



- Divide memory up into small pages
- Map virtual pages to physical pages
 - Each process has separate mapping (PT)
- Allow OS to gain control on certain operations
 - Read-only pages trap to OS on write
 - Invalid pages trap to OS on read or write
 - OS can change mapping and resume application

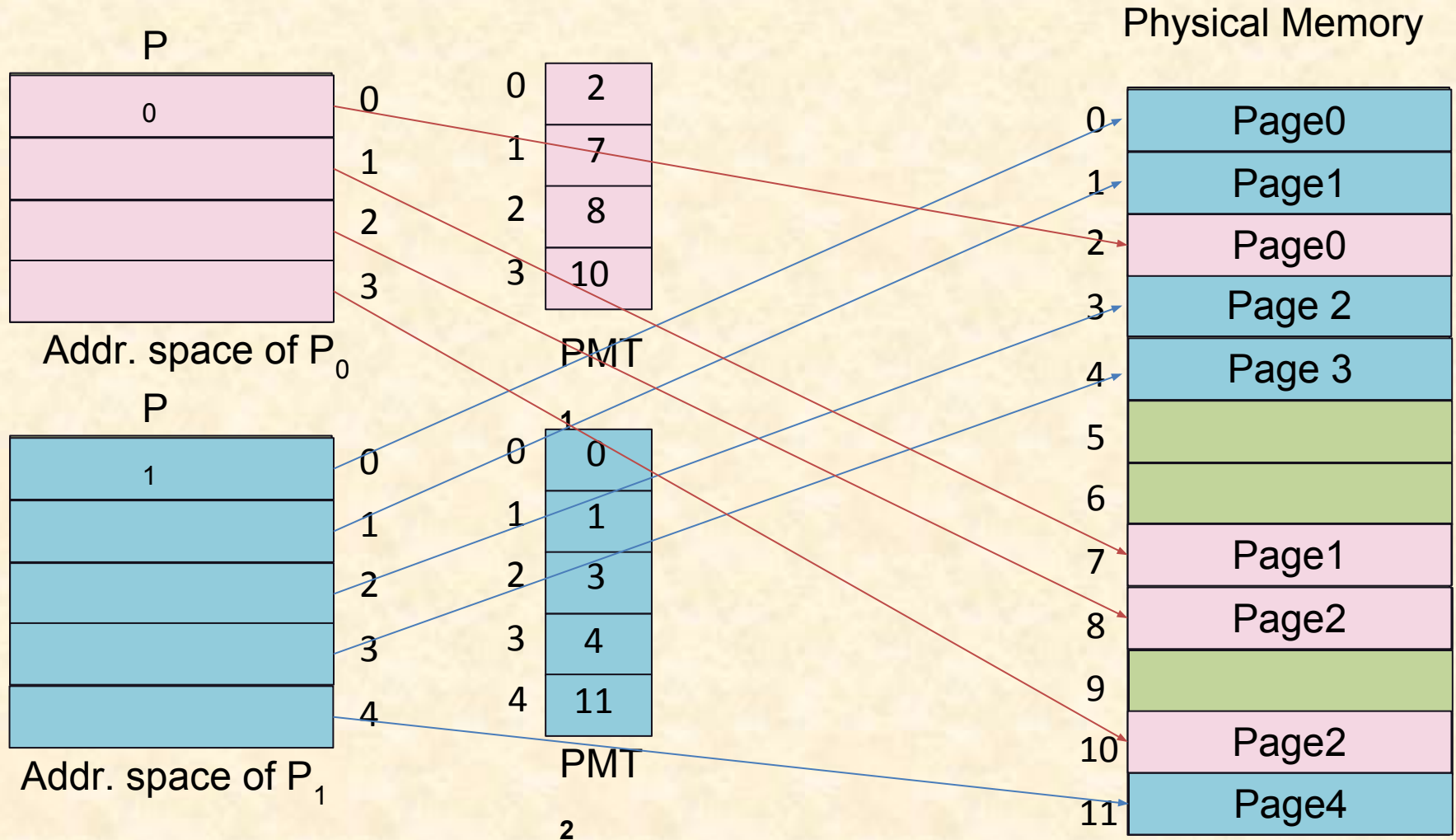
Paging

Memory partitions are of equal size



OS allocates page table for each process

Multiple Process in Paging



No need to have contiguous allocation

Pros: No external fragmentation

Cons: Complex Mapping

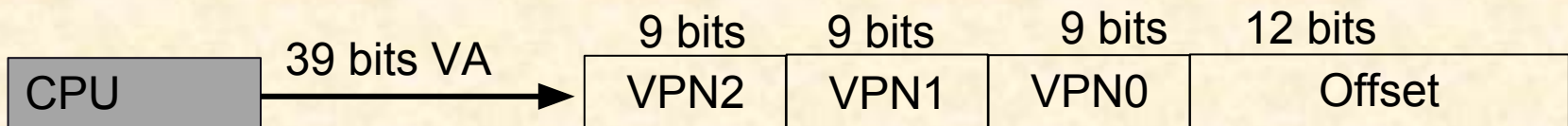
Internal fragmentation approx. 50% of a page

RISC-V virtual memory

- Supports different addressing modes:
- Sv32, Sv39, Sv48 (32, 39, 48-bit virtual address)
- RiscV xv6 uses Sv39 addressing mode

Sv39 virtual memory

- If paging turned on: virtual address is 39 bits
- Page size 4KB (offset 12 bits)

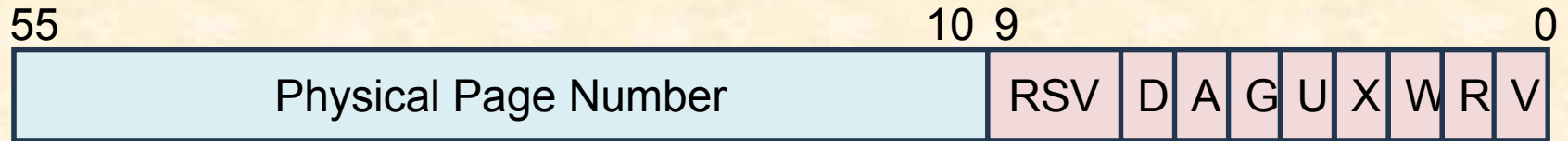


Virtual Address (39 bits):

- Bits [63–39]: **Sign Extension** — Bits 38 to 63 must be identical (copies of bit 38) to ensure canonical addresses.
- Bits [38–30]: **VPN[2]** (9 bits) — Level-2 page table index.
- Bits [29–21]: **VPN[1]** (9 bits) — Level-1 page table index.
- Bits [20–12]: **VPN[0]** (9 bits) — Level-0 page table index.
- Bits [11–0]: **Page Offset** (12 bits) — Index within the 4 KiB target page.

Page table entries (PTE)

- Page entries are of 56 bits

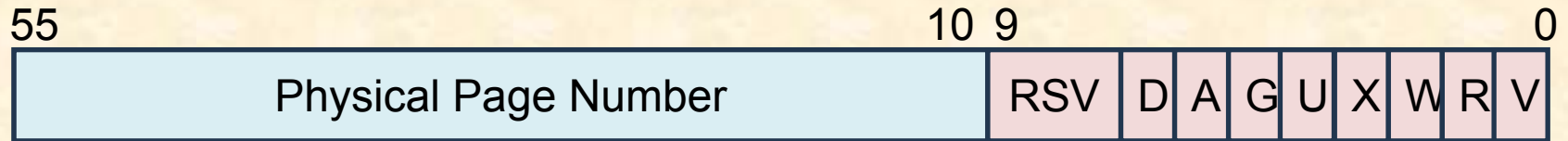


Physical Address (56 bits):

- Bits [55–12]: **PPN** (44 bits) — Physical Page Number.
- Bits [11–0]: **Page Offset** (12 bits) — Passed directly from the virtual address without modification.

Page table entries (PTE)

- Page entries are of 56 bits



Some important bits:

- U=1: If set, user space can access this virtual address
U=0: Kernel page. Access only is supervisor mode
- W: If set, the CPU can write to this virtual address
- V: If set, an entry for this virtual address exists
- RSV: Ignored by MMU

Sv39 3-level Paging

